DOCKER MASTERY

Level 2 — Part 2

Production Images & Interview Mastery

A complete reproduction of the second half of your Level 2 journey. Topics 4, 5, and 6 covered in full depth — multi-stage builds, .dockerignore, and image optimization for production. Plus a comprehensive 40-question interview prep section ranked from basic to advanced, with structured answers ready to deliver.

Every question you asked is preserved. Every explanation is reproduced. Designed so you never need to look anywhere else.

Before You Start

This is Part 2 of your Level 2 journey. Part 1 covered the foundations — writing Dockerfiles, understanding layers and caching, the CMD vs ENTRYPOINT distinction. With those locked in, you're ready for the techniques that separate junior engineers from seniors.

What changes in Part 2

Part 1 was about *making things work*. Part 2 is about *making things production-grade*. Same code, same app, same goal — but the engineer who finishes Part 2 ships images that are 10x smaller, more secure, faster to deploy, and easier to operate.

How to use this document

Read top to bottom. Don't skip the hands-on exercises. The questions you originally asked during the journey are preserved as boxed "Your Question" sections so you can see what triggered each deep-dive. The interview prep section at the end is what you should rehearse out loud before any Docker interview.

> **KEY DIFFERENCE FROM PART 1**
>
> Part 1 taught you what each instruction does. Part 2 teaches you **how to compose them under constraints** — size budget, security requirements, build speed targets, observability needs. This is the level interviewers test for senior roles.

TOPIC 04

Multi-stage Builds

the secret to tiny production images · build vs run foundation · all languages

What problem does this solve?

Right now, when you build an image for a compiled or build-heavy app, you have a problem you might not even realize you have:

THE PROBLEM

Your final image is full of stuff your app doesn't actually need to run.

To *build* a Node app you need: Node.js + npm, all dev dependencies (TypeScript compiler, webpack, eslint, jest), source code (.ts files, tests, configs), build tools (gcc, python, make for native modules).

To *run* the same Node app you only need: Node.js, production dependencies, the compiled JS output.

Same pattern for every language

Java needs Maven and JDK to build, only JRE to run. Go needs the compiler to build, only the binary to run. Python with C-extensions needs gcc to compile, only the runtime to execute. **Build dependencies and runtime dependencies are different sets.** A single-stage build mixes them and ships everything to production.

Visualizing the bloat

FINAL IMAGE — 1.2 GB (single-stage)	ACTUALLY NEEDED — 180 MB
node + npm runtime (needed)	node runtime
production deps (needed)	production deps
compiled JS output (needed)	compiled JS output
dev deps: typescript, jest (BLOAT)	
source .ts files, tests (BLOAT)	
gcc, python, make (BLOAT)	

a single-stage image vs what your app actually uses

LOCK THIS IN

Build dependencies and runtime dependencies are different sets. A single-stage build mixes them — and ships everything to production. Multi-stage fixes this.

YOUR QUESTION #DEEP

I don't understand. What is building and what is running? Explain this concept deeper because this is the foundation.

The single sentence to anchor everything

THE ANCHOR

Build = creating the package. Run = executing the package.

These are not two phases of one thing. They are two completely different actions, separated in time, that happen on potentially different machines, by different people, with different tools available.

The factory and the customer

Imagine you make instant noodles.

At the factory (BUILD time):

Workers operate giant industrial machines. Massive boilers, dough mixers, dryers, packaging robots. Quality control labs, testing equipment. Raw flour, salt, oil — in huge sacks. The factory itself is enormous, hot, loud, expensive. **The factory's job: turn raw ingredients into a sealed packet of noodles.**

At the customer's home (RUN time):

You, in your kitchen. A kettle of hot water. A bowl. The sealed packet. **Your job: open the packet, add water, eat.**

THE QUESTION

Does the customer need an industrial dough mixer in their kitchen to eat noodles? **Obviously not.** The dough mixer was needed to *create* the noodles. To *eat* them, you only need the noodles themselves and hot water.

But here's what happens with a single-stage Dockerfile: **you ship the entire factory to the customer's house with the noodles.**

Apply this to software

Suppose you wrote a TypeScript web server. The file you wrote is `server.ts`:

```
// server.ts — what YOU wrote
import express, { Request, Response } from 'express';

const app = express();
app.get('/', (req: Request, res: Response) => {
  res.send('Hello!');
});
app.listen(3000);
```

Critical fact: your computer cannot run this file directly. Node.js doesn't understand TypeScript. It only understands JavaScript. Before this app can ever start, someone has to **convert it** to JavaScript. That conversion is called **building**. It uses the TypeScript compiler (tsc) to read server.ts, strip out type annotations, and produce server.js. **This server.js is the "noodle packet".** It's the finished product. Node can execute it directly.

What each phase needs:

Phase	What it needs	Why
BUILD	TypeScript compiler, type definitions, source .ts files, tsc binary, possibly webpack, possibly babel, ESLint, dev d	To convert TS → JS
RUN	Node.js runtime, express package, the compiled server.js — about 30 MB	To execute the JavaScript

The TypeScript compiler is **only needed once**, at build time, to produce server.js. After that, it's useless. It's like keeping the dough mixer in your kitchen forever just because you ate one packet of noodles.

Why "build" and "run" feel like the same thing in Docker

In Docker, you run two commands:

```
docker build -t myapp .   # ← BUILD: creates the image
docker run myapp          # ← RUN: starts the container
```

These are two separate commands at two separate times. But **inside docker build**, Docker also runs commands. Things like `RUN npm install`. The word "RUN" inside the Dockerfile is misleading — it doesn't mean "run the app." It means "run this command *during the build process*."

> **UNTANGLE THE VOCABULARY**
>
> **The RUN lines in your Dockerfile happen during docker build.** They prepare the image. They produce files. They install things. They are **build-time activities**. After build, they're done forever.
>
> **The CMD line happens during docker run.** It starts your actual application. It's the **runtime activity**. It happens every time the container starts.

Trace through what happens in time

```
FROM python:3.11-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install -r requirements.txt    ← build time
COPY src/ src/                          ← build time
CMD ["python", "src/app.py"]            ← run time
```

9:00 AM — you type:

```
docker build -t myapp .
```

For 30 seconds, Docker pulls the base image, creates /app, copies requirements.txt, **executes pip install** (downloads Flask, runs setup scripts, installs files), copies your src/ folder, saves the result as an image tagged `myapp`.

9:00:30 AM — build is done. What exists now?

An image sitting on your disk. **That's it. Nothing is running. No Python process exists. Your app is not alive.**

Pip install ran. It already happened. It's a memory now. The result of pip install (the installed packages) is baked into the image. But pip is not running anymore. **Nothing is running.**

9:05 AM — five minutes later, you start the app:

```
docker run -d -p 8080:5000 myapp
```

Now something happens: Docker creates a container from the image. It executes the CMD. A Python process starts, opens port 5000, begins serving requests. Your app is alive.

9:05 PM — twelve hours later, second container:

```
docker run -d -p 8081:5000 myapp
```

Two containers running, both from the same frozen image. Both started their own Python process. The image hasn't changed. The build didn't happen again. **One build, many runs.**

> **THE CENTRAL INSIGHT**
>
> **Build happens once. Run happens many times.**
>
> Once the image is built, the tools used to build it are not needed anymore. Pip already finished installing. The TypeScript compiler already produced JavaScript. But Docker, in a single-stage build, doesn't know this. It keeps everything in the image because it can't tell which files were "build helpers" and which are "things the running app needs."

Three quick tests to check it clicked

Test 1. I run docker build and it succeeds. I check docker ps. What do I see?
→ *Nothing. Building doesn't start a container.*

Test 2. My Dockerfile has RUN pip install flask. After docker build, is pip still running inside the image somewhere?
→ *No. Pip ran during build, finished, and exited. Flask files are baked into the image. Pip itself is still installed (you could call it again at runtime), but it is not running. Nothing is running until docker run.*

Test 3. Why is the TypeScript compiler "build-time only" but Node.js itself is "runtime"?
→ *Because tsc converts .ts → .js once, and after that conversion it's never used again. But Node.js executes .js files — needed every time a request comes in.*

Make it tangible — try this on your machine

```
mkdir build-vs-run && cd build-vs-run

cat > Dockerfile << 'EOF'
FROM python:3.11-slim
WORKDIR /app
RUN echo "I ran at BUILD time" > /tmp/build-log.txt
RUN pip install flask
COPY . .
CMD ["sh", "-c", "echo 'I ran at RUN time'; cat /tmp/build-log.txt; sleep 30"]
EOF

# 1. BUILD — watch the RUN commands execute
docker build -t demo .

# 2. After build, verify nothing is running
docker ps          # empty

# 3. RUN — now the CMD executes
docker run --rm demo
# You see "I ran at RUN time" + the build-log file

# 4. Run AGAIN — same image, new container, build did NOT re-run
docker run --rm demo
```

WHEN IT CLICKS

Build messages appear once (during step 1). CMD message appears every time you docker run. **That asymmetry is the whole concept made visible.**

Idea 2 — The mental model: two kitchens

Now that build vs run is locked in, this analogy will land. Imagine you're catering a dinner party. You don't bring your **prep kitchen** to the party — the cutting boards, raw ingredients, mixing bowls, dirty pans, the recipe book. You bring the **finished dishes**, plated.

Multi-stage builds work exactly the same:

- **Stage 1 = the prep kitchen.** Big, messy, full of build tools. Compiles your code, installs dev dependencies, runs tests, produces the final artifact.
- **Stage 2 = the dinner party.** Clean, minimal. Has only the finished food. You **copy the plated dish** from the prep kitchen, leave the mess behind. That's what gets shipped.

> **THE WHOLE CONCEPT**
>
> When the build finishes, **only the final stage becomes your image**. Everything from earlier stages is discarded — the cutting boards, the raw ingredients, the entire prep kitchen. **Gone.** You only ship what's needed at runtime.

Idea 3 — The syntax: multiple FROMs

A multi-stage Dockerfile is just a regular Dockerfile with **multiple FROM lines**. Each FROM starts a new stage. You name stages with `AS <name>` so you can refer back to them.

```
# ■■ STAGE 1: builder (the prep kitchen) ■■
FROM node:20 AS builder
WORKDIR /app
COPY package*.json ./
RUN npm install              # installs ALL deps (dev + prod)
COPY . .
RUN npm run build            # produces /app/dist

# ■■ STAGE 2: production (the dinner party) ■■
FROM node:20-slim
WORKDIR /app
COPY package*.json ./
RUN npm install --production # only PROD deps
COPY --from=builder /app/dist ./dist
CMD ["node", "dist/server.js"]
```

Three things to notice:

1. Two FROM lines = two stages. The first is the heavy build environment. The second is the slim runtime.

2. The named stage. `FROM node:20 AS builder` gives this stage a name. Without it, you'd have to use stage indexes (`--from=0`), which break the moment you reorder.

3. The magic line: COPY --from=builder /app/dist ./dist. This copies a folder *from a previous stage* into the current stage. Source files, dev deps, the entire builder stage — none of it crosses over. Only /app/dist does.

> **THE ENDING**
>
> When Docker finishes, **only the last FROM becomes your image.** The builder stage was used to produce /app/dist, then thrown away.

Idea 4 — Three real-world patterns

Pattern A — Go (the cleanest example)

Go compiles to a single static binary. The runtime image needs *literally nothing* else — not even a shell.

```
FROM golang:1.22 AS builder
WORKDIR /src
COPY go.mod go.sum ./
RUN go mod download
COPY . .
RUN CGO_ENABLED=0 go build -o /bin/server ./cmd/server

FROM gcr.io/distroless/static-debian12
COPY --from=builder /bin/server /server
CMD ["/server"]
```

Builder image: ~900 MB (full Go toolchain). **Final image:** ~20 MB. The final image has no shell, no package manager, no OS utilities — just your binary.

Pattern B — Node.js / TypeScript

```
FROM node:20 AS builder
WORKDIR /app
COPY package*.json ./
RUN npm ci                          # installs ALL deps including TypeScript
COPY . .
RUN npm run build                   # tsc compiles to /app/dist

FROM node:20-slim
WORKDIR /app
COPY package*.json ./
RUN npm ci --omit=dev               # only production deps
COPY --from=builder /app/dist ./dist
USER node
CMD ["node", "dist/server.js"]
```

Why two npm install? The first one needs typescript, webpack, eslint, jest to *build*. The second only needs the runtime libraries.

Pattern C — Python with C extensions

Most relevant to you. Some Python packages (numpy, pandas, psycopg2, cryptography) need a C compiler to install. You don't want gcc and 200 MB of dev headers in your production image.

```
FROM python:3.11 AS builder
WORKDIR /app
RUN apt-get update && apt-get install -y --no-install-recommends \
    gcc python3-dev libpq-dev \
 && rm -rf /var/lib/apt/lists/*
COPY requirements.txt .
RUN pip wheel --no-cache-dir --wheel-dir /wheels -r requirements.txt

FROM python:3.11-slim
WORKDIR /app
COPY --from=builder /wheels /wheels
COPY requirements.txt .
RUN pip install --no-cache-dir --no-index --find-links=/wheels -r requirements.t
xt \
 && rm -rf /wheels
COPY src/ src/
RUN useradd -m -u 1000 appuser && chown -R appuser:appuser /app
USER appuser
EXPOSE 5000
CMD ["gunicorn", "--workers=2", "--bind=0.0.0.0:5000", "src.app:app"]
```

Stage 1 has gcc and dev headers, builds pre-compiled wheel files into /wheels/. Stage 2 starts fresh from slim, copies the wheels over, installs them with `--no-index --find-links=/wheels`. **No gcc in your final image. No dev headers. Just the compiled .so files.**

Idea 5 — Copy from anywhere

Multi-stage builds aren't limited to two stages. You can have as many as you want. And `COPY --from` doesn't just take stage names — it can also copy from **any image on Docker Hub**.

```
# Use a specific version of curl from another image
COPY --from=curlimages/curl:latest /usr/bin/curl /usr/local/bin/curl

# Use kubectl from the official Kubernetes image
COPY --from=bitnami/kubectl:1.29 /opt/bitnami/kubectl/bin/kubectl /usr/local/bin
/kubectl
```

You don't `RUN apt-get install curl`. You **steal the binary** from a known-good image. This is how senior engineers compose tooling images — assembling them from other images like Lego blocks.

Three-stage pattern

```
FROM node:20 AS deps
WORKDIR /app
COPY package*.json ./
RUN npm ci

FROM node:20 AS builder
WORKDIR /app
COPY --from=deps /app/node_modules ./node_modules
COPY . .
RUN npm run build

FROM node:20-slim
WORKDIR /app
COPY --from=deps /app/node_modules ./node_modules
COPY --from=builder /app/dist ./dist
CMD ["node", "dist/server.js"]
```

Change source code, only builder and final stage rebuild. Change package.json, all three rebuild.

Idea 6 — Targeting a stage with --target

```
# Build only up to the "builder" stage — useful for tests
docker build --target builder -t myapp:test .
docker run --rm myapp:test npm test

# Build everything — production image
docker build -t myapp:prod .
```

One Dockerfile can serve dev, test, and prod without three separate files. Just point --target at the stage you want.

```
FROM node:20 AS builder

FROM builder AS dev
ENV NODE_ENV=development
CMD ["npm", "run", "dev"]

FROM node:20-slim AS prod
CMD ["node", "dist/server.js"]
```

Idea 7 — Convert YOUR Dockerfile

Your real Dockerfile from earlier:

```
FROM python:3.11-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY src/ src/
EXPOSE 5000
CMD ["gunicorn", "--workers=2", "--bind=0.0.0.0:5000", "src.app:app"]
```

> **BRUTAL HONESTY**
>
> For a pure-Python Flask app with simple dependencies, this is *fine*. Multi-stage gives you the biggest win when you have C-extension packages (numpy, psycopg2, cryptography, lxml). If your requirements.txt is just Flask + gunicorn + requests — pure Python — multi-stage barely helps. **Don't add complexity for ceremony.**

But the moment you add: psycopg2, numpy, pandas, lxml, cryptography, pillow, mysqlclient — you're paying for build tooling you don't need.

Idea 8 — Common mistakes

Mistake 1 — Forgetting --from

```
COPY /app/dist ./dist          # ■ tries to copy from build context
COPY --from=builder /app/dist ./dist   # ✓ copies from previous stage
```

Mistake 2 — Numeric stage references

```
COPY --from=0 /app/dist ./dist # ■ works, but fragile
COPY --from=builder /app/dist ./dist  # ✓ explicit, doesn't break
```

Mistake 3 — CMD in the wrong stage

Only the **final stage's** CMD/ENTRYPOINT/USER/EXPOSE/WORKDIR ends up in your image. Anything in earlier stages is build-only.

Mistake 4 — Bad caching order in stages

Same caching rule as Topic 2 — package.json first, then `COPY . .`. Multi-stage doesn't free you from layer caching discipline.

Idea 9 — Hands-on experiment

```
mkdir multistage-test && cd multistage-test
mkdir src

cat > src/server.js << 'EOF'
const http = require('http');
http.createServer((req, res) => {
  res.end('Hello!\n');
}).listen(3000);
EOF

cat > package.json << 'EOF'
{"name":"demo","main":"src/server.js","dependencies":{}}
EOF

# Single-stage version
cat > Dockerfile.single << 'EOF'
FROM node:20
WORKDIR /app
COPY package.json .
RUN npm install
COPY . .
CMD ["node", "src/server.js"]
EOF

# Multi-stage version
cat > Dockerfile.multi << 'EOF'
FROM node:20 AS builder
WORKDIR /app
COPY package.json .
RUN npm install
COPY . .

FROM node:20-slim
WORKDIR /app
COPY --from=builder /app .
CMD ["node", "src/server.js"]
EOF

docker build -f Dockerfile.single -t demo:single .
docker build -f Dockerfile.multi -t demo:multi .

docker images | grep demo
# demo  multi   ~240 MB
# demo  single  ~1.1 GB
```

Idea 10 — When NOT to use multi-stage

PUSHING BACK ON THE TREND

It's not always the right call.

Skip multi-stage when:

- Pure interpreted apps with no build step. A simple Flask app with only Python dependencies has no "build artifact" to extract.
- Development images where speed matters more than size.
- CI/CD tooling, not deployment. A test runner image needs all the tools.

Use multi-stage when:

- Compiled languages (Go, Rust, Java, C++).
- TypeScript / transpiled languages.
- Python with C extensions.
- You want one Dockerfile for dev/test/prod via --target.

THE HONEST TEST

Does your RUN step produce something you can copy out, while leaving the rest behind? If yes, multi-stage helps. If no, it's just ceremony.

Idea 11 — Self-check

Q1. What does COPY --from=builder /app/dist ./dist do?

→ Copies /app/dist from the previous stage named builder into the current stage's ./dist.

Q2. When the build finishes, what gets shipped?

→ Only the last stage. Everything from earlier stages is discarded.

Q3. Why use a separate builder stage?

→ To keep build tools out of the final runtime image.

Q4. What does docker build --target builder do?

→ Stops the build at the builder stage. Useful for tests.

Q5. Can you COPY --from from external images?

→ Yes. COPY --from=alpine:3.19 /etc/alpine-release /info works.

The whole topic in one paragraph

TOPIC 4 — THE WHOLE THING

Multi-stage builds let you write a Dockerfile with multiple FROM lines — each starts a new build "stage." Earlier stages do the heavy work. Later stages are slim runtime images that use COPY --from=<stage> to grab only the finished artifacts. When the build completes, only the **last stage** becomes your image. This separates *build dependencies* from *runtime dependencies*, often shrinking final images from 1+ GB down to under 200 MB.

TOPIC 05

.dockerignore

the file you didn't know you needed · secret leakage prevention · clean builds

This is the shortest topic in Level 2 — but skipping it is one of the most common mistakes. A missing .dockerignore quietly makes every build slower, leaks secrets into images, and bloats files that should be tiny.

Idea 1 — The problem you don't know you have

Remember from Topic 1 — when you run docker build, the first thing Docker does is **pack your entire current folder into a tarball** and ship it to the daemon. That tarball is the **build context**.

The Docker daemon doesn't have access to your laptop's filesystem. It only sees what you sent it. So **everything in your build context** is what COPY . . is allowed to grab from.

Run this in your project root:

```
du -sh .
```

You might be horrified. A "small Flask app" can easily be 500 MB once you count .git, node_modules, .venv, __pycache__ folders, log files, IDE settings.

THE PROBLEM

Without a .dockerignore, ALL of that gets shipped to the Docker daemon every time you build. And depending on your Dockerfile, much of it might end up *inside* your final image.

What's actually in your build context

NO .dockerignore — 487 MB shipped	WITH .dockerignore — 80 KB shipped
src/ — the only thing you wanted	src/ — the actual application code
.git/ — 240 MB — entire repo history	requirements.txt — dependency list
node_modules/ — 180 MB — gets reinstalled	Dockerfile — the recipe
.venv/ — 50 MB — won't even work in container	
.env — 1 KB — YOUR SECRETS	
__pycache__/ — 12 MB scattered everywhere	
tests/ · logs/ · docs/ · build artifacts	

.vscode/ · .idea/ · .DS_Store · IDE junk	

without .dockerignore your entire folder ships every build

Idea 2 — What .dockerignore actually is

.dockerignore is a plain text file you put in the same directory as your Dockerfile. It works exactly like .gitignore — list patterns, Docker excludes anything matching those patterns from the build context.

> **THE WHOLE CONCEPT**
>
> **That's it.** No magic. Just a list of "don't ship this" patterns.

```
# .dockerignore
.git
node_modules
.env
__pycache__
*.pyc
.venv/
*.log
```

Files matching .dockerignore **never get packed into the tarball.** They never reach the daemon. They cannot end up in your image — even if you COPY . ., they were excluded *before* the COPY ran.

Idea 3 — The three problems it solves

Problem 1 — Slow builds

Every docker build starts with "transferring context". With 500 MB, 5–10 seconds *before any actual build work begins*. With 80 KB, milliseconds.

```
=> [internal] load build context                                    0.0s
=> => transferring context: 487.34MB                                ← yours might be
huge
```

If "transferring context" shows MB or GB, your .dockerignore is missing or broken.

Problem 2 — Bloated images via COPY . .

Without .dockerignore, you're potentially copying:

- A 240 MB .git folder — your entire commit history.
- A 180 MB node_modules — about to be wiped out and reinstalled, but it sat in a layer first.
- All your test fixtures, build artifacts, IDE config.

This bloats the image and wastes layer cache. Layers are immutable — even if you remove the files later, the bytes are in the layer regardless.

Problem 3 — Secret leakage (the dangerous one)

```
.env
SECRET_API_KEY=sk_live_a1b2c3d4e5f6
DATABASE_URL=postgresql://admin:password123@db.prod.com/mydb
```

You write COPY . . in your Dockerfile. Now the .env file is **permanently baked into your image**:

```
docker run --rm myapp:v1 cat /app/.env
# SECRET_API_KEY=sk_live_a1b2c3d4e5f6
```

REAL INCIDENT PATTERN

Once a secret is in any layer of any pushed image, it cannot be unleaked. Even if you push a new version, the old image with the secret is still in registries, in caches, in backups. A .dockerignore with .env and .env.* would have prevented it entirely.

Idea 4 — The syntax

```
# Comments start with hash
# Match files by exact name
.env

# Match by extension
*.log
*.pyc

# Match a folder anywhere in the tree
node_modules
__pycache__

# Match a folder only at the root
/build

# Recursive wildcard — match at any depth
**/*.tmp

# Negation — keep this even if a previous pattern excluded it
*.md
!README.md       # exclude all .md files EXCEPT README.md
```

Memorize these four patterns

- **name** matches anywhere in the tree (file or folder)
- **/name** matches only at the root
- ***.ext** matches all files with that extension
- ****/pattern** is recursive — at any depth

Idea 5 — A production-quality template

```
# ■■ Version control ■■
.git
.gitignore

# ■■ Environments and secrets ■■
.env
.env.*
*.pem
*.key
secrets/

# ■■ Python ■■
__pycache__
*.pyc
*.egg-info
.venv
venv/
dist/
build/
.pytest_cache
.coverage
.mypy_cache
.ruff_cache

# ■■ Node ■■
node_modules
npm-debug.log*
.next
.cache

# ■■ IDE ■■
.vscode
.idea
*.swp
.DS_Store
Thumbs.db

# ■■ Logs and temp ■■
*.log
logs/
tmp/

# ■■ Documentation ■■
docs/
*.md
!README.md

# ■■ Tests (optional, in production) ■■
tests/
*_test.py

# ■■ Docker itself ■■
Dockerfile*
.dockerignore
docker-compose*.yml
```

Idea 6 — The corner cases that bite

Corner case 1 — matched against build context root

```
docker build -f services/api/Dockerfile -t myapp .
                                                ↑
                                                build context is "."
```

The .dockerignore in the **root** applies — not one inside services/api/. **Patterns are evaluated relative to the build context, not the Dockerfile.**

Corner case 2 — Negation is order-sensitive

```
*.md           # exclude all .md files
!README.md     # but keep this one
```

If you reverse the order, README.md will be excluded with the rest. **The ! rule must come after the pattern it's overriding.**

Corner case 3 — context vs final image

.dockerignore controls what goes into the **build context**. It does not control what's in your final image directly — that's still controlled by your COPY instructions.

Idea 7 — How to verify it's working

Method 1 — Watch the build context size

```
docker build -t test .
```

Look at `=> => transferring context: 487.34MB`. After fixing it, should drop dramatically.

Method 2 — Inspect the image directly

```
docker run --rm -it myapp:v1 sh
ls -la /app
ls -la /app/.env  # should fail — file shouldn't exist
exit
```

Idea 8 — Hands-on experiment

```
mkdir dockerignore-test && cd dockerignore-test
mkdir src
echo "print('hello')" > src/app.py
echo "Flask==3.0.0" > requirements.txt

# Simulate junk
mkdir -p .git/objects && dd if=/dev/zero of=.git/junk bs=1M count=50 2>/dev/null
mkdir -p node_modules && dd if=/dev/zero of=node_modules/junk bs=1M count=100 2>
/dev/null
echo "SECRET=supersecret" > .env

cat > Dockerfile << 'EOF'
FROM python:3.11-slim
WORKDIR /app
COPY . .
CMD ["python", "src/app.py"]
EOF

# Build WITHOUT .dockerignore
docker build -t test:bloated .
# transferring context: ~150MB

docker run --rm test:bloated cat /app/.env
# SECRET=supersecret  ← shipped into the image

# Now add .dockerignore
cat > .dockerignore << 'EOF'
.git
node_modules
.env
*.log
EOF

docker build -t test:clean .
# transferring context: ~1KB

docker run --rm test:clean cat /app/.env
# cat: /app/.env: No such file or directory  ← excluded!
```

Idea 9 — Common mistakes

Mistake 1 — Forgetting it exists. Every new project, the first file alongside the Dockerfile should be .dockerignore. Make this a reflex.

Mistake 2 — Excluding too much and breaking the build. If you exclude something your COPY needs, the build fails with a "file not found in build context" error.

Mistake 3 — Trusting .dockerignore to keep secrets out of git. It doesn't. Use both .gitignore and .dockerignore.

Mistake 4 — Adding .dockerignore to .dockerignore. Don't. Docker needs to read it.

Idea 10 — Self-check

Q1. Does .dockerignore exclude files from the image or from the build context?

→ Files from the build context. They can't be COPY'd, so they indirectly stay out of the image too.

Q2. If your Dockerfile only has COPY src/ src/, does .dockerignore matter?

→ Less, but yes. It still affects build context size and is a safety net if someone adds COPY . . later.

Q3. Where does .dockerignore need to be located?

→ At the root of the build context — the same folder you point docker build at.

Q4. What's the most dangerous file to forget excluding?

→ .env and any file with secrets. They get baked in permanently and can't be unleaked once pushed.

The whole topic in one paragraph

> **TOPIC 5 — THE WHOLE THING**
>
> .dockerignore is a plain text file in your build context root that tells Docker which files and folders to **exclude when packing up the build context** to send to the daemon. Syntax is nearly identical to .gitignore. Without it, every docker build ships your entire project folder including .git, node_modules, .env files — slowing builds, bloating images, and leaking secrets. **Always create a .dockerignore alongside every Dockerfile — make it a reflex.**

TOPIC 06

Image Optimization for Production

the senior-engineer playbook · five dimensions of production-ready

This is the topic that brings Level 2 together. Everything you've learned converges here into a single discipline: **shipping images that are small, fast, secure, and observable.**

Idea 1 — What "production-ready" means

Most beginners think "my image runs the app, so it's done." A senior engineer thinks across **five dimensions**:

Dimension	What it means	Why it matters
1. Size	small image, slim base + multi-stage	faster pulls, faster autoscale
2. Security	non-root, minimum packages, no secrets	smaller attack surface, fewer CVEs
3. Build speed	cache-friendly layer order, tight context	fast CI, fast iteration
4. Reliability	graceful shutdown, healthchecks, pinned versions	predictable production behavior
5. Observability	stdout logs, structured JSON, OCI labels	diagnose issues at 3 AM

LOCK THIS IN

Production-ready isn't one thing. It's five things at once.

Idea 2 — The seven techniques to reduce image size

When an interviewer asks "how do you reduce Docker image size," here are seven techniques ordered by impact. Reasoning matters more than the trick itself.

Technique 1 — Pick a smaller base image

Variant	Size	When to use
python:3.11	~900 MB	almost never in production
python:3.11-slim	~45 MB	the sweet spot — recommended
python:3.11-alpine	~17 MB	smallest, but C-extensions can break

One decision saves 95% of image size before you add anything. Same pattern for every language — node:20 → node:20-slim, openjdk:21 → eclipse-temurin:21-jre-alpine.

Technique 2 — Multi-stage builds

Covered fully in Topic 4. Build deps and runtime deps are different. Stage 1 has the heavy toolchain. Stage 2 copies only the artifact. For Go: 900 MB → 20 MB.

Technique 3 — Chain RUN commands

Layers are immutable. Deleting a file in a later layer doesn't actually free space. Combine commands with && so cleanup happens before the layer is committed.

```
# ■ Bad — three layers, apt cache permanent
RUN apt-get update
RUN apt-get install -y curl
RUN rm -rf /var/lib/apt/lists/*

# ✓ Good — one layer, cleanup before commit
RUN apt-get update && \
    apt-get install -y --no-install-recommends curl && \
    rm -rf /var/lib/apt/lists/*
```

Technique 4 — Use a .dockerignore file

Covered in Topic 5. Smaller build context, smaller image, no leaked secrets.

Technique 5 — Clean package manager caches

```
# Python
RUN pip install --no-cache-dir -r requirements.txt

# Node
RUN npm ci --omit=dev && npm cache clean --force

# Apt
RUN apt-get install -y --no-install-recommends ca-certificates && \
    rm -rf /var/lib/apt/lists/* /var/cache/apt/*

# Apk (Alpine)
RUN apk add --no-cache curl
```

Technique 6 — Install only what production needs

```
# Node
RUN npm ci --omit=dev

# Python — split requirements.txt and requirements-dev.txt
```

Technique 7 — Distroless or scratch (advanced)

Covered deeply below — most candidates won't mention this. Mentioning it signals senior-level depth.

Idea 3 — The unified production Dockerfile

All five dimensions in one file. The senior-engineer reference template for a Python Flask app:

```
# syntax=docker/dockerfile:1.6
FROM python:3.11.9-slim-bookworm AS builder

ENV PYTHONDONTWRITEBYTECODE=1 \
    PYTHONUNBUFFERED=1 \
    PIP_NO_CACHE_DIR=1

RUN apt-get update && \
    apt-get install -y --no-install-recommends \
        gcc python3-dev libpq-dev && \
    rm -rf /var/lib/apt/lists/*

WORKDIR /app
COPY requirements.txt .
RUN pip install --prefix=/install --no-cache-dir -r requirements.txt

FROM python:3.11.9-slim-bookworm AS runtime

ENV PYTHONDONTWRITEBYTECODE=1 \
    PYTHONUNBUFFERED=1 \
    PORT=5000

RUN apt-get update && \
    apt-get install -y --no-install-recommends libpq5 && \
    rm -rf /var/lib/apt/lists/*

RUN groupadd -r app --gid=1000 && \
    useradd -r -g app --uid=1000 --create-home --shell=/sbin/nologin app

WORKDIR /app
COPY --from=builder /install /usr/local
COPY --chown=app:app src/ src/
USER app
EXPOSE 5000

HEALTHCHECK --interval=30s --timeout=5s --start-period=10s --retries=3 \
    CMD ["python", "-c", "import urllib.request; urllib.request.urlopen('http://
localhost:5000/health', timeout=3)"]

LABEL org.opencontainers.image.title="myapp" \
      org.opencontainers.image.version="1.0.0" \
      org.opencontainers.image.source="https://github.com/me/myapp" \
      org.opencontainers.image.licenses="MIT"

CMD ["gunicorn", "--workers=2", "--bind=0.0.0.0:5000", \
     "--access-logfile=-", "--error-logfile=-", "src.app:app"]
```

What each piece achieves:

- **PYTHONUNBUFFERED=1** — logs flush immediately to stdout

- **Build deps in builder only** — gcc never touches runtime image
- **--prefix=/install** — clean path to copy out cleanly
- **libpq5 (runtime) instead of libpq-dev (build)** — smaller, more secure
- **Non-root user before CMD** — UID 1000, /sbin/nologin shell
- **COPY --chown=app:app** — sets ownership in same step, saves a layer
- **HEALTHCHECK** — Kubernetes/Docker uses this
- **OCI LABELs** — traceability back to source git commit
- **--access-logfile=-** — gunicorn logs to stdout

Idea 4 — The pin-everything discipline

Look closely at the FROM line:

```
FROM python:3.11.9-slim-bookworm
```

Not python:3.11. Not python:3.11-slim. The full version: 3.11.9-slim-bookworm.

Why? Because python:3.11 floats. Tomorrow it might point to 3.11.10. **You'll rebuild your image and silently get a different OS underneath your app.**

```
# requirements.txt — always pin
flask==3.0.2
gunicorn==21.2.0
psycopg2-binary==2.9.9
```

For ultimate paranoia, pin the base image by **digest**:

```
FROM python:3.11.9-slim-bookworm@sha256:7e61c0ad2f7ba2c8e9d2fa7bff4c4f0c1d8e3...
```

A tag can be re-pointed. A digest cannot.

Idea 5 — Logging discipline

> **THE RULE**
>
> **Containers should log to stdout and stderr. Never to files inside the container.**

The wrong way:

```
logging.basicConfig(filename='/var/log/myapp.log', level=logging.INFO)
```

When the container dies, the logs die with it.

The right way:

```
import logging
import sys

logging.basicConfig(
    stream=sys.stdout,
    level=logging.INFO,
    format='%(asctime)s %(levelname)s %(name)s %(message)s'
)
```

Docker captures stdout. docker logs myapp shows them. Kubernetes log aggregators collect automatically. **Observability for free.**

Bonus: structured JSON logging

Plain text:

```
2024-05-06 10:23:14 INFO Order processed for user 42
```

JSON (searchable by field):

```
{"ts":"2024-05-06T10:23:14Z","level":"INFO","msg":"Order processed","user_id":42
}
```

Idea 6 — HEALTHCHECK

```
HEALTHCHECK --interval=30s --timeout=5s --start-period=10s --retries=3 \
    CMD curl -f http://localhost:5000/health || exit 1
```

The flags:

- --interval=30s — check every 30 seconds
- --timeout=5s — fail if check takes more than 5s
- --start-period=10s — give app 10s to start
- --retries=3 — fail unhealthy after 3 consecutive failures

Your app needs a /health endpoint:

```
@app.route('/health')
def health():
    return {'status': 'ok'}, 200
```

> **KUBERNETES NOTE**
>
> In Kubernetes specifically, HEALTHCHECK is usually replaced by livenessProbe and readinessProbe. Both are valid.

Idea 7 — Metadata labels (OCI standards)

```
LABEL org.opencontainers.image.title="myapp" \
      org.opencontainers.image.version="1.0.0" \
      org.opencontainers.image.source="https://github.com/me/myapp" \
      org.opencontainers.image.revision="a1b2c3d4" \
      org.opencontainers.image.licenses="MIT"
```

Why this matters:

- **Traceability** — given any image SHA, find the exact git commit it was built from.
- **Tooling** — registries, scanners, dashboards read these labels.
- **Compliance** — many enterprises require source labels.

In CI inject dynamically:

```
docker build \
  --label org.opencontainers.image.revision=$(git rev-parse HEAD) \
  --label org.opencontainers.image.created=$(date -u +%Y-%m-%dT%H:%M:%SZ) \
  -t myapp:1.0.0 .
```

Idea 8 — Vulnerability scanning

trivy — by Aqua Security:

```
trivy image myapp:1.0.0
```

Output looks like:

```
myapp:1.0.0 (debian 12.5)
Total: 7 (LOW: 4, MEDIUM: 2, HIGH: 1, CRITICAL: 0)
```

docker scout — built into Docker Desktop:

```
docker scout cves myapp:1.0.0
docker scout recommendations myapp:1.0.0
```

The discipline:

1. Scan every image in CI before deployment.
2. Fail the build on HIGH or CRITICAL vulnerabilities.
3. When CVE shows up, upgrade the package or document an explicit exception.

SENIOR-ENGINEER RULE

No image with a known CRITICAL CVE goes to production without a documented exception. Set a CI gate.

YOUR QUESTION #DEEP

Technique 7 — Distroless or scratch (advanced) — I did not understand this. Go deeper.

You wanted a clean, deeper explanation in simple terms.

The single sentence to anchor this

THE ANCHOR

Distroless = a base image stripped down so aggressively that it doesn't even contain a Linux distribution anymore. Just your app's runtime. Nothing else.

The name is the clue. **Dist-ro-less = without a distribution.** No Debian. No Ubuntu. No Alpine. No "operating system" in the way you usually think of one.

What's actually inside a normal Linux image

When you write FROM ubuntu:22.04 or FROM python:3.11-slim, you're inheriting **way more stuff than you realize**:

- **A shell** (/bin/sh, /bin/bash) — so you can docker exec

- **Package manager** (apt, dpkg) — to install software
- **File utilities** — ls, cat, cp, mv, rm, find, grep
- **Network tools** — curl, wget, ping, netstat, dig
- **Process tools** — ps, top, kill
- **Text editors** — vi, nano (sometimes)
- **System libraries** — libc, libssl, hundreds of .so files
- **The Python runtime** — python3, pip
- **Your application code**

> **HONEST QUESTION**
>
> When your Flask app is running and serving requests, which of these does it actually use? **Just the bottom two.** The Python runtime and your code.
>
> Everything else is there in case **a human** wants to log in and poke around. **All those other utilities are baggage. Useful for humans, useless for the app, dangerous in production.**

Why distroless came to exist

Google built distroless and open-sourced it in 2017. Two problems drove it.

Problem 1: every package is a potential vulnerability.

When a security researcher finds a bug in bash, every container with bash is potentially exploitable. Even if your app doesn't use bash, the bash binary is *still vulnerable*. The only defense is to remove bash. Same for curl, apt, every binary.

Problem 2: attackers love your shell.

When an attacker exploits a bug in your application, the very first thing they try to do is **get a shell**. Why? Because once they have a shell, they can:

- Use curl to download attack tools
- Use apt to install hacking software
- Use cat /etc/passwd to enumerate users
- Use ls, find, grep to look for valuable files
- Use nc (netcat) to open backdoor connections

Without a shell, almost none of this works. The exploit might still get them initial code execution, but they can't *do* anything with it. So Google asked: **what if we just... didn't include any of that stuff?** The answer was distroless.

What you give up — be honest

1. You can't docker exec to debug.

```
docker exec -it myapp bash
# Error: executable file not found in $PATH
docker exec -it myapp sh
# Error: executable file not found in $PATH
docker exec -it myapp ls
# Error: executable file not found in $PATH
```

Nothing works. There's nothing to exec. You can't get inside.

2. Logs become your only window.

Your application *must* log everything important — startup state, errors, slow queries, key events.

3. Health checks need the language runtime, not curl.

```
# In a normal image:
HEALTHCHECK CMD curl -f http://localhost:8080/health || exit 1

# In distroless (no curl):
HEALTHCHECK CMD ["python", "-c", "import urllib.request; urllib.request.urlopen(
'http://localhost:8080/health')"]
```

4. Debug variants exist as a workaround.

Google publishes :debug variants — gcr.io/distroless/python3-debian12:debug — which add busybox. Use these in development, regular ones in production.

What you gain — concretely

1. Smaller attack surface.

Typical CVE scan of python:3.11-slim finds dozens of known vulnerabilities. Distroless typically has 0–3 because there's almost nothing to be vulnerable.

2. Smaller image size.

For Go and Rust, dramatic. A Go binary on golang:1.22 ends up around 900 MB. Same binary on gcr.io/distroless/static is **10 MB. 90× smaller.**

For Python and Node, modest. python:3.11-slim is ~45 MB. gcr.io/distroless/python3 is ~50 MB. **Nearly the same** — the runtime itself dominates.

> **INTERVIEW DETAIL**
>
> Distroless saves a lot of size for **compiled** languages (Go, Rust, Java), much less for **interpreted** languages (Python, Node).

3. Forced discipline.

Junior engineers hate this. Senior engineers see it as a feature. Without docker exec as escape hatch, your team is forced to log properly, set up real observability, not SSH into production.

scratch — the absolute extreme

If distroless is "minimal," scratch is "nothing." A FROM scratch image is **completely empty**. Just your binary.

```
FROM golang:1.22 AS builder
WORKDIR /src
COPY . .
RUN CGO_ENABLED=0 go build -o /server

FROM scratch
COPY --from=builder /server /server
CMD ["/server"]
```

Final image size: just the size of your binary. A small Go web server might be 8 MB total.

FROM scratch only works for **statically compiled languages** — Go, Rust, C++ static. Does NOT work for Python or Node.

> **THE RULE**
>
> **scratch for compiled languages, distroless for interpreted languages.**

Try it on your machine — see for yourself

```
mkdir distroless-test && cd distroless-test

cat > app.py << 'EOF'
import http.server, socketserver
with socketserver.TCPServer(("", 8000), http.server.SimpleHTTPRequestHandler) as
 httpd:
    print("Serving on 8000")
    httpd.serve_forever()
EOF

cat > Dockerfile.slim << 'EOF'
FROM python:3.11-slim
WORKDIR /app
COPY app.py .
CMD ["python", "app.py"]
EOF

cat > Dockerfile.distroless << 'EOF'
FROM gcr.io/distroless/python3-debian12
WORKDIR /app
COPY app.py .
CMD ["app.py"]
EOF

docker build -f Dockerfile.slim -t demo:slim .
docker build -f Dockerfile.distroless -t demo:distroless .

# Try to shell into each
docker run -d --name slim-test demo:slim
docker exec -it slim-test bash    # Works! You're inside.
exit

docker run -d --name distroless-test demo:distroless
docker exec -it distroless-test bash
# Error: executable file not found in $PATH
docker exec -it distroless-test sh
# Error: executable file not found in $PATH
# THERE IS NO SHELL. This is distroless.
```

THE MOMENT IT CLICKS

When you see "executable file not found" three times trying to get a shell — that's when distroless becomes real. There's nothing inside.

Idea 9 — Reproducibility

Same source code → same image, every time. Several things break this:

1. Floating base image tags — pin them.
2. Floating package versions in apt-get.
3. Network-dependent steps — use lock files.
4. Build args injected dynamically — like git SHA labels.
5. Timestamps embedded in builds.

> **THE GOAL**
>
> Given a specific git SHA + a specific Dockerfile, you can rebuild a byte-identical image months later. For most teams, "pinned + locked" solves 90% of reproducibility problems.

Idea 10 — The optimization checklist

Pre-deploy review checklist for any image:

Size

- ■ Slim or distroless base image
- ■ Multi-stage build separates build deps from runtime
- ■ .dockerignore excludes everything unnecessary
- ■ Package manager caches cleaned in same RUN
- ■ Only production dependencies installed in final stage

Security

- ■ Non-root USER set before CMD
- ■ No secrets in image (verify with docker history)
- ■ Base image pinned (specific version, ideally digest)
- ■ Minimum packages installed
- ■ Image scanned with trivy/scout in CI

Build speed

- ■ COPY ordered: deps first, source last
- ■ Build context size < 10 MB
- ■ BuildKit enabled

Reliability

- Exec-form CMD (JSON array)
- HEALTHCHECK present (or Kubernetes probes)
- App handles SIGTERM gracefully
- Versions pinned in requirements.txt / package-lock.json

Observability

- App logs to stdout/stderr (not files)
- Logs are structured JSON when possible
- OCI labels set (version, revision, source)
- HEALTHCHECK endpoint exists in app

> **THE BENCHMARK**
>
> Most teams hit 60–70% of these. Hitting 95%+ is what separates senior engineers.

Idea 11 — Tools to know

`dive`

visualizes layer-by-layer what's eating space.

`hadolint`

Dockerfile linter. Catches anti-patterns. Run in CI.

`docker-slim`

automated image shrinking. Can shrink 1 GB → 30 MB.

`syft`

generates Software Bill of Materials (SBOM).

Idea 12 — Common production anti-patterns

✗ Latest tag in production

FROM python:latest floats. Pin specific versions.

✗ Running as root

Always create and use a non-root user.

✗ Logging to files

Logs disappear with the container. Use stdout.

✗ Storing data inside the container

Lost on restart. Use volumes.

✗ Multiple processes per container

One container = one process.

✗ Credentials in ENV in Dockerfile

Visible in docker history forever. Inject at runtime.

✗ No HEALTHCHECK and no probes

Container appears running but is dead.

Level 2 Graduation Summary

WHAT YOU CAN NOW CLAIM

You can write a Dockerfile that follows industry best practices. You understand every instruction. You order layers for optimal caching. You use multi-stage builds when they help and skip them when they don't. You ship images that are slim, secure, and observable. You can decode any docker build output line by line. You know where Docker stores data on disk. You can answer the interview question "how do you reduce image size" with seven techniques and reasoning. **That's a senior-junior level of Docker fluency. Most engineers years into their career don't have this depth.**

INTERVIEW PREP

40 Questions on Topics 4, 5, 6

from basic to advanced · structured answers · what to memorize

The mindset before you answer

When an interviewer asks "how do you reduce Docker image size," they're testing three things at once:

1. Do you know multiple techniques (not just one)?
2. Can you reason about WHY each works?
3. Do you know which to apply first (priority and impact)?

HOW TO USE THIS SECTION

Read each question, understand the structure, then close this and try to deliver it in your own words. **Memorized answers crumble at the first follow-up question.** Internalize the structure, not the exact words.

Section 1 — Multi-stage Builds (Topic 4)

BASIC

Q1 · BASIC

What is a multi-stage build?

The structured answer:

"A multi-stage build is a Dockerfile pattern where you have multiple FROM instructions, each starting a new build stage. Earlier stages do the heavy work — compiling code, installing build dependencies, running tests. Later stages are slim runtime images that copy only the final artifacts forward using COPY --from=<stage>. When the build finishes, only the last stage becomes your image; everything else is discarded. The point is to separate build dependencies from runtime dependencies so you don't ship the entire build toolchain to production."

Q2 · BASIC

Why would you use a multi-stage build instead of a single-stage one?

The structured answer:

"Three main reasons. First, smaller final image — for compiled languages like Go, you can drop from 900 MB to under 20 MB by leaving the toolchain behind. Second, smaller attack surface — fewer packages in production means fewer potential CVEs. Third, separation of concerns — your Dockerfile clearly distinguishes 'build steps' from 'runtime steps,' which makes it easier to reason about and modify."

WATCH OUT FOR THE TRAP

If they ask 'but doesn't that make builds slower?' — the answer is no. BuildKit can run independent stages in parallel and the cache helps. Don't fall for the trap.

Q3 · BASIC

What does COPY --from=builder /app/dist ./dist do?

The structured answer:

"It copies the /app/dist folder from a previous build stage named builder into the current stage's ./dist directory. The --from flag is what makes this cross-stage — without it, COPY only sees files from the build context on your host. This is the bridge that lets a runtime stage pick up artifacts produced by a builder stage."

BONUS POINT

Mention that --from= can also reference an external image like --from=alpine:3.19 to copy files from any image on a registry. Senior-level detail.

Q4 · BASIC

What gets shipped as the final image — all stages, or just the last one?

The structured answer:

“Only the last stage. Everything before it is used during the build to produce intermediate artifacts, but those layers are discarded. That's why the technique works — you can have a 1 GB builder stage and still end up with a 50 MB final image, because nothing in the builder stage is in the final image except what you explicitly copied forward.”

INTERMEDIATE

Q5 · INTERMEDIATE

Walk me through writing a multi-stage Dockerfile for a Node.js TypeScript app.

The structured answer:

"Two stages. The first uses node:20 as the builder, installs all dependencies including dev dependencies needed for compilation like TypeScript and webpack, copies the source, and runs npm run build to produce the compiled JavaScript in /app/dist. The second stage uses node:20-slim, installs only production dependencies with npm ci --omit=dev, copies just the /app/dist folder from the builder, and sets the CMD. The point is the builder has tools you only need to compile TypeScript — those don't need to ship to production."

```
FROM node:20 AS builder
WORKDIR /app
COPY package*.json ./
RUN npm ci
COPY . .
RUN npm run build

FROM node:20-slim
WORKDIR /app
COPY package*.json ./
RUN npm ci --omit=dev
COPY --from=builder /app/dist ./dist
USER node
CMD ["node", "dist/server.js"]
```

Q6 · INTERMEDIATE

Can you have more than two stages? When would you use that?

The structured answer:

"Yes, you can have any number. A common three-stage pattern is dependencies → builder → runtime. The dependencies stage installs node_modules once. The builder stage uses those modules to compile the code. The runtime stage takes both. The benefit is more granular caching: if you only change source code, only the builder and runtime stages rebuild — the dependencies stage stays cached. If you only change package.json, all three rebuild."

Q7 · INTERMEDIATE

What's the difference between FROM x AS builder and just FROM x?

The structured answer:

"The AS builder part gives the stage a name. Without it, you'd refer to the stage by its index — --from=0 for the first stage, --from=1 for the second. Named stages are better practice because if you reorder stages or insert new ones, named references still work. Numeric references break silently. Always use names."

Q8 · INTERMEDIATE

Can you use COPY --from to copy from images that aren't in your Dockerfile?

The structured answer:

“Yes. COPY --from=alpine:3.19 /etc/alpine-release /info would copy a file from a fresh Alpine image without you having to build that stage yourself. This is useful for grabbing specific binaries from known-good images — like pulling kubectl from bitnami/kubectl instead of installing it manually. It's how senior engineers compose images from existing ones rather than building everything from scratch.”

ADVANCED

Q9 · ADVANCED

How does layer caching work across multiple stages?

The structured answer:

"Each stage has its own independent layer cache, exactly as a single-stage build would. The caching rules from Topic 2 apply within each stage — line-text changes invalidate that line and everything below it. Across stages, the boundary is COPY --from=<stage>. If the source stage rebuilds, the file you're copying might be different, which invalidates the COPY in the destination stage. So change in stage 1 can cascade into stage 2 only through what you actually copy across. An important nuance: BuildKit can build independent stages in parallel."

Q10 · ADVANCED

What's --target and when would you use it?

The structured answer:

"docker build --target <stage> stops the build at a specific stage instead of going all the way to the end. The use case is one Dockerfile serving multiple environments. You can have a dev stage with hot-reload, a test stage that runs your test suite, and a prod stage with the slim runtime. Then docker build --target dev for development, --target test in CI, and the default for production. One source of truth instead of three Dockerfiles."

Q11 · ADVANCED

Should every Dockerfile use multi-stage builds?

The structured answer:

"No. Multi-stage builds are valuable when you have build dependencies that aren't needed at runtime — compilers, dev tools, build artifacts you transform. But for a pure Python Flask app with only runtime dependencies, multi-stage adds complexity without meaningful savings. The honest test is: does your build produce an artifact you can copy out, leaving the toolchain behind? If yes, multi-stage helps. If no — like a simple interpreted-language app — a slim base image alone is enough."

WHY THIS IMPRESSES

You pushed back on the implicit assumption that multi-stage is always better. Senior engineers know when not to apply techniques.

Q12 · ADVANCED

What's the difference between using a multi-stage build vs just using a smaller base image?

The structured answer:

“Different problems. A smaller base image — like switching from python:3.11 to python:3.11-slim — reduces what's already in the image before you add anything. Multi-stage reduces what you add during the build. They compose: use a slim base image as your runtime stage, AND use multi-stage to keep the heavy toolchain in the builder stage. Together they get you the smallest possible images. Using one but not the other leaves savings on the table.”

Section 2 — .dockerignore (Topic 5)

BASIC

Q13 · BASIC

What is .dockerignore?

The structured answer:

“It's a plain text file in the root of your build context that tells Docker which files and folders to exclude when packing the build context to send to the daemon. The syntax is nearly identical to .gitignore. Files matching those patterns never reach the daemon, which means they can't end up in the image even if you COPY . .”

Q14 · BASIC

Why is .dockerignore important?

The structured answer:

“Three reasons. First, build speed — without it, you ship your entire project including .git, node_modules, and other junk, slowing the upload to the daemon. Second, image size — COPY . . would otherwise pull in everything, bloating layers. Third, security — without it, files like .env containing secrets can be silently baked into your image and exposed to anyone who pulls it.”

Q15 · BASIC

What's the syntax for .dockerignore?

The structured answer:

*“Same as .gitignore. One pattern per line. * is a wildcard for one path segment, ** is recursive across any depth. Lines starting with # are comments. Patterns can be negated with ! to override a previous exclusion — for example, *.md followed by !README.md excludes all markdown except the README.”*

INTERMEDIATE

Q16 · INTERMEDIATE

What should a typical .dockerignore contain?

The structured answer:

“Universal categories: version control metadata like .git, language-specific dependency folders like node_modules or .venv, language-specific cache folders like __pycache__, environment files containing secrets like .env and .env., IDE settings like .vscode and .idea, build outputs like dist/ and build/ if regenerated inside the image, log files, OS metadata like .DS_Store, and test artifacts. Generally, anything that's regenerable, transient, or sensitive should be excluded.”*

Q17 · INTERMEDIATE

If your Dockerfile uses specific COPY paths like COPY src/ src/, do you still need .dockerignore?

The structured answer:

“Less critical, but yes. .dockerignore still affects build context upload speed — even if you don't COPY a file, it's still being shipped to the daemon. Plus it's a safety net: if anyone later adds a COPY . . to the Dockerfile, your .dockerignore already prevents leaks. Defense in depth.”

Q18 · INTERMEDIATE

Can .dockerignore keep secrets out of your image?

The structured answer:

“It can keep secrets out of new images you build. But if you've already built and pushed an image containing the secret, .dockerignore doesn't help retroactively — that image is still in the registry, in pull caches, in backups, and the secret is exposed in docker history. The right approach is preventive: have .env in .dockerignore from day one. And critically, never put secrets in ENV instructions in the Dockerfile — use runtime injection via docker run -e, Docker secrets, or Kubernetes Secrets.”

WHY THIS IMPRESSES

You went beyond the question to discuss the broader secret-handling discipline. Unprompted depth differentiates senior candidates.

Q19 · INTERMEDIATE

What's the difference between .gitignore and .dockerignore?

The structured answer:

“Similar syntax, completely different purpose. .gitignore controls what gets committed to git. .dockerignore controls what gets sent to the Docker daemon. They're independent files for

independent tools. A file in .gitignore but not in .dockerignore could still end up in your image. You usually want both — there's typically significant overlap, but neither is a substitute for the other."

ADVANCED

Q20 · ADVANCED

How do you verify your .dockerignore is working correctly?

The structured answer:

“Three ways. First, watch the 'transferring context:' line in the build output — if it shows MB or GB for a small project, your .dockerignore is missing files. Second, after building, run the image and inspect the filesystem with docker run --rm -it myimage sh and ls /app to confirm sensitive files aren't there. Third, use docker history to see what got into each layer.”

Q21 · ADVANCED

If a file is in the build context but not in any COPY command, does it end up in the image?

The structured answer:

“No, but with caveats. A file has to be explicitly COPY'd in to end up in any image layer. So if you only COPY src/ src/, files outside src/ aren't copied. However, the file is still being uploaded to the daemon, which slows the build, and there's no defense if someone later adds a COPY . . The principle is: .dockerignore controls what enters the build process at all, while COPY controls what lands in the image.”

Q22 · ADVANCED

Where does .dockerignore live, and what happens if you have multiple Dockerfiles?

The structured answer:

“It lives at the root of the build context — the directory you pass as the last argument to docker build. Patterns are evaluated relative to that root. If you have multiple Dockerfiles in different subdirectories and build each from its own folder, each can have its own .dockerignore. But if you build them all from a parent directory using -f services/api/Dockerfile, the root .dockerignore applies — not one inside services/api/.”

Section 3 — Image Optimization (Topic 6)

BASIC

Q23 · BASIC

How do you reduce Docker image size?

THIS IS THE BIG ONE

Worth memorizing the structure. Most candidates give 10 seconds and say 'use a smaller base image.' This puts you in the top 5%.

The structured answer:

"Several techniques, ordered by impact. The two biggest wins: first, choose a smaller base image — switching from python:3.11 to python:3.11-slim drops you from 900 MB to 45 MB before you add anything. Second, use multi-stage builds to keep build tooling out of the runtime image; for compiled languages this can mean a 50× reduction. Beyond those: chain RUN commands with && so cleanup happens within the same layer — Docker layers are immutable, so deleting a file in a later layer doesn't actually free space. Add a .dockerignore to keep junk out of the build context. Use package-manager flags like pip install --no-cache-dir, npm ci --omit=dev. Install only production dependencies in the final stage. For the smallest possible images, use distroless or scratch base images. In practice for a new Dockerfile I'd start with slim base + multi-stage + .dockerignore. Those three usually take you from 1+ GB down to under 200 MB."

Q24 · BASIC

Why should containers run as non-root?

The structured answer:

"Defense in depth. If your application has a vulnerability, the attacker now has whatever privileges your app process has. If you're running as root, the attacker is root inside the container. With certain misconfigurations like privileged mode or sensitive volume mounts, that can escalate to root on the host. Running as a non-root user limits the blast radius. The fix is two lines: create a user with RUN useradd ... and switch to it with USER appuser."

Q25 · BASIC

Why use a slim base image instead of the default one?

The structured answer:

"The default image variants like python:3.11 come with a full Linux distribution including documentation, locales, build tools, and dozens of utilities your app doesn't need. A slim variant strips those out, keeping only what's necessary to run the language runtime. For Python, that's a 95% reduction — from about 900 MB to 45 MB. Slim is the right default for almost

everything."

INTERMEDIATE

Q26 · INTERMEDIATE

Why does chaining RUN commands actually save space?

The structured answer:

“Because Docker layers are immutable. Once a layer is written, it cannot be modified — only added on top of. So if layer A creates a file and layer B deletes it, the file is still in layer A taking up space. The deletion in layer B is just a 'whiteout' marker that hides the file at runtime, but the bytes never leave layer A. Chaining commands with && puts them in the same RUN, which means they share one filesystem snapshot, and the cleanup happens before the layer is committed.”

Q27 · INTERMEDIATE

What's the difference between slim and alpine base images?

The structured answer:

“Both are stripped-down variants but they take different approaches. Slim is based on Debian and uses glibc as its C library — familiar, broadly compatible, most packages just work. Alpine is much smaller because it uses musl as its C library and BusyBox utilities instead of GNU. The size difference is significant. The tradeoff is musl isn't 100% binary-compatible with glibc; some Python C-extension packages either fail to install or behave subtly differently on Alpine. I default to slim and switch to Alpine only after testing.”

Q28 · INTERMEDIATE

What's HEALTHCHECK and why does it matter?

The structured answer:

“It's a Dockerfile instruction that tells Docker how to test if your container is alive and serving correctly. The check command runs periodically inside the container — if it exits 0, the container is healthy. If it consistently fails, Docker marks the container unhealthy and orchestrators like Docker Swarm restart it. The typical command is something like curl -f http://localhost:5000/health against a /health endpoint your app exposes. In Kubernetes specifically, the in-Dockerfile HEALTHCHECK is usually replaced by Kubernetes's own livenessProbe and readinessProbe.”

Q29 · INTERMEDIATE

What does LABEL do, and why use it?

The structured answer:

“Labels are key-value metadata attached to the image. They don't change runtime behavior — they're for tools and humans to query. The Open Container Initiative defines a standard set like org.opencontainers.image.source, org.opencontainers.image.revision, and org.opencontainers.image.version. The practical value is traceability: given any image SHA

running in production, you can find the exact git commit it was built from. In CI you'd inject them dynamically with docker build --label org.opencontainers.image.revision=$(git rev-parse HEAD).”

Q30 · INTERMEDIATE

Why log to stdout instead of files?

The structured answer:

“Three reasons. One: when the container dies, files inside it die too — your logs are gone, often right when you need them most. Two: stdout is captured by Docker's logging driver, which integrates with everything — docker logs, Kubernetes log aggregators, Datadog, Splunk, ELK. Logs flow into your observability stack automatically. Three: it follows the twelve-factor-app principle of treating logs as event streams. Containers should be ephemeral and stateless; persistent logs belong in external systems.”

ADVANCED

Q31 · ADVANCED

Walk me through optimizing this bloated Dockerfile.

They might give you something like:

```
FROM python:3.11
WORKDIR /app
COPY . .
RUN apt-get update
RUN apt-get install -y gcc
RUN pip install -r requirements.txt
RUN apt-get install -y curl
EXPOSE 5000
CMD python src/app.py
```

The structured answer:

"First, FROM python:3.11 is the full Debian image — about 900 MB. I'd switch to python:3.11-slim, dropping that to 45 MB. Second, COPY . . happens before pip install, which means any source-code change invalidates the dependency layer's cache. Should be split: COPY requirements.txt ., RUN pip install, then COPY . . Third, the four separate RUN commands each create their own layer. The apt-get cache from line 4 is permanently in the image. I'd combine them with && and add rm -rf /var/lib/apt/lists/. Fourth, pip install is missing --no-cache-dir, leaving the pip download cache. Fifth, no USER instruction — the container runs as root. I'd add a non-root user. Sixth, the CMD is in shell form — should be exec form: CMD ["python", "src/app.py"]. And there's no .dockerignore, no HEALTHCHECK, no labels. The optimized version would be a multi-stage build with build deps in a builder stage, slim runtime, non-root user, exec-form CMD, proper layer ordering. Image size would drop from 1.2 GB to under 100 MB."*

CHEF'S-KISS ANSWER

You're touching every Level 2 topic in one walkthrough. If you can deliver something close to this, you'll seriously impress.

Q32 · ADVANCED

What is distroless?

The structured answer:

"Distroless is a class of base images from Google that strip out everything except the language runtime and your app — no shell, no package manager, no curl, no ls. The idea came from observing that most of what's in a typical Linux image exists for humans to debug, not for the application to run. The wins are smaller attack surface and smaller image size. Without a shell, attackers who get code execution can't easily pivot to use post-exploitation tools. CVE counts also drop dramatically. The cost is you can't docker exec to debug interactively. Distroless requires good logging and observability. The biggest savings are for compiled languages — a Go app on distroless can be 10 MB instead of 900 MB. For pure binaries, you can go even

further with FROM scratch, completely empty, just your binary."

Q33 · ADVANCED

How do you handle secrets in Docker images?

The structured answer:

“Never bake them in. Several approaches in order of increasing strength. The wrong way is ENV API_KEY=xyz in the Dockerfile or COPY .env . — those put the secret in a layer permanently, visible via docker history. The basic right way is runtime injection: docker run -e API_KEY=xyz myapp or --env-file .env. The secret is in the running container's environment but never in the image. Combined with .dockerignore listing .env, this is the minimum. Better is using a secrets manager — AWS Secrets Manager, HashiCorp Vault, or Kubernetes Secrets. The container fetches secrets at startup or has them mounted as files. For BuildKit specifically, --mount=type=secret lets you use a secret during build without baking it in. Whatever the approach, the rule is: secrets are a runtime concern, not a build-time concern.”

Q34 · ADVANCED

How would you scan a Docker image for vulnerabilities?

The structured answer:

“The standard tools are trivy from Aqua Security, grype from Anchore, and docker scout built into Docker Desktop. Usage: trivy image myapp:1.0.0 produces a report listing every CVE found in OS packages and language dependencies. The discipline is to make scanning a CI gate. Every image build runs a scan, and the build fails if HIGH or CRITICAL vulnerabilities are found. Multi-stage builds and distroless help here directly — fewer packages means fewer CVEs to track.”

Q35 · ADVANCED

What does it mean for a Docker build to be reproducible?

The structured answer:

“A reproducible build means the same source code and the same Dockerfile produce a byte-identical image, every time, regardless of when or where it's built. Several things break reproducibility. Floating tags like python:3.11 change meaning over time. RUN apt-get install pulls whatever package version is current. Network requests during build can return different artifacts. Timestamps embedded in builds. The fixes: pin base images by version or even by digest with @sha256:..., lock dependency versions. For most teams, 'pinned + locked' is reproducible enough.”

Section 4 — Cross-cutting / Trick Questions

These don't fit one topic but show up often. They test whether you understand connections between topics.

Q36 · ADVANCED

If I add a .dockerignore after I've already built and pushed an image, does it retroactively remove files from the published image?

The structured answer:

"No. .dockerignore only affects future builds. Files already in a published image are permanent — removing them from a future build doesn't touch the previously pushed image. To 'remove' something from a published image, you build a new version without it and push it as a new tag. The old image with the secret still exists in the registry until explicitly deleted. This is why you should never push an image with a secret. Once it's pushed, even briefly, assume it's compromised forever. Rotate the secret."

Q37 · ADVANCED

You see a Dockerfile with RUN apt-get install -y curl followed several lines later by RUN rm /usr/bin/curl. Will the final image have curl in it?

The structured answer:

"No, you won't be able to run curl from a shell — but the curl binary's bytes are still on disk in the layer where it was installed. Layers are immutable; the rm in a later layer just creates a whiteout marker that hides curl from the filesystem view. The bytes remain, taking up space, and could in theory be recovered by anyone with access to the image's individual layers. For real removal, the install and the removal need to be in the same RUN command."

WHAT THIS TESTS

Whether you actually understand layers, not just 'the user can't see curl.' Most beginners say 'no, it's removed.' A senior engineer says 'no, but it's still in the layer.'

Q38 · INTERMEDIATE

What's the practical difference between EXPOSE 5000 and docker run -p 8080:5000?

The structured answer:

"EXPOSE is purely documentation — it doesn't actually open or map any port. It tells humans and tools 'this image listens on port 5000.' -p 8080:5000 actually maps host port 8080 to container port 5000, making the app reachable from outside. You always need -p to expose a port to the host. EXPOSE is a hint; -p is the action."

Q39 · INTERMEDIATE

Why use npm ci instead of npm install in a Dockerfile?

The structured answer:

“Three reasons. First, npm ci is faster — it installs straight from package-lock.json without resolving dependency versions. Second, it's deterministic — the same package-lock.json always gives you the same node_modules, which matters for reproducibility. Third, it fails if package.json and package-lock.json are out of sync — preventing the silent drift you'd get from npm install.”

Q40 · ADVANCED

How do you build a Docker image that works on both x86 and ARM?

The structured answer:

“Build a multi-arch image using docker buildx. The command is roughly docker buildx build --platform linux/amd64,linux/arm64 -t myapp:1.0 --push . This creates a manifest list with one image per architecture, pushed under a single tag. When users pull the image, the registry serves them the version matching their architecture. For this to work, your base images must also be multi-arch — most official images like python, node, golang already are. In CI, you can build multi-arch images either with QEMU emulation (slow but simple) or by running native builders for each architecture (fast).”

How to actually prepare

Reading these answers isn't preparation. **Saying them out loud is preparation.**

Exercise 1 — The 60-second drill

Pick five questions at random. Set a timer for 60 seconds per question. Without notes, deliver an answer. Record yourself if you can. Listen back. Where do you stumble? Those are the gaps.

Exercise 2 — The follow-up game

After answering a question, ask yourself 'what would the interviewer follow up with?' Then answer that. Then the follow-up to the follow-up. Senior interviewers drill three or four levels deep. If you stop at the first answer, you're a junior. If you have three levels of depth ready, you're a senior.

Exercise 3 — The inverse exercise

Given an answer, can you reconstruct the question? This forces you to understand the structure of how senior engineers explain things, not just the content.

The five answers worth memorizing word-for-word

Most answers should be in your own words. But these five are so commonly asked, and so structured, that having them rehearsed gives you confidence:

1. Q23 — How do you reduce image size? (the seven techniques)
2. Q9 — How does layer caching work across stages?
3. Q31 — Walk me through optimizing this Dockerfile.
4. Q32 — What's distroless?
5. Q33 — How do you handle secrets?

> **THE BENCHMARK**
>
> If you can deliver these five answers at senior level, the interviewer will form a strong impression and most other questions become softballs.

Final piece of advice

> **WHEN YOU DON'T KNOW**
>
> When an interviewer asks something you don't know, **don't bullshit.** Say 'I haven't worked with that specifically, but based on what I know about [related concept], I'd guess...' That's a senior move. Confidently making things up is a junior move that experienced interviewers spot instantly. **Honesty + reasoning under uncertainty is what they're actually testing.**

LEVEL 2 COMPLETE

■ You've gone from 'I can write a Dockerfile that runs my app' to 'I can engineer production images that pass senior code review.' Take a beat to recognize that. It's not nothing. When you're ready, move to Level 3 — but first run the experiments, convert one of your real apps, push it. **Use it before you build on top of it.**